République Tunisienne

Ministère de l'Enseignement Supérieur et de la Recherche Scientifique

École Nationale d'Ingénieurs de Sfax

ENIS

Département Génie Informatique

RAPPORT DE PROJET DE FIN D'ANNÉE

PFA – 2024/2025

Conception et Développement d'une Plateforme E-Learning Intelligente avec Moteur de Recommandation par IA

Réalisé par :

- **BAHLOUL Donia**
- **FRIKHA Amin**
- **ABDELHEDI Wassim**

Encadrant : [Nom de l'encadrant]

Établissement : ENIS

Spécialité : Génie Informatique

Année universitaire : 2024 – 2025

Dédicaces

À nos familles et proches,

qui nous ont soutenus tout au long de ce parcours académique.

À nos enseignants,

pour leur dévouement et leur accompagnement précieux.

À tous ceux qui croient en la puissance de l'éducation numérique.

BAHLOUL Donia, FRIKHA Amin, ABDELHEDI Wassim

Remerciements

Nous tenons à exprimer notre profonde gratitude à toutes les personnes qui ont contribué à la réalisation de ce projet de fin d'année.

Nous adressons tout d'abord nos sincères remerciements à notre encadrant, pour son soutien indéfectible, ses conseils avisés et sa disponibilité tout au long de ce travail. Sa rigueur académique et son expertise technique nous ont été d'une valeur inestimable.

Nous remercions également l'ensemble du corps enseignant du département informatique pour la qualité de la formation dispensée, qui nous a fourni les bases solides nécessaires à l'accomplissement de ce projet.

Nos remerciements s'étendent également à nos familles pour leur patience, leur encouragement et leur soutien moral, sans lesquels ce travail n'aurait pas été possible.

Enfin, nous remercions tous nos camarades de promotion pour l'entraide et l'esprit d'équipe qui ont caractérisé notre parcours de formation.

Résumé

Ce rapport présente le travail réalisé dans le cadre du Projet de Fin d'Année (PFA), portant sur la conception et le développement d'une plateforme e-learning intelligente baptisée **PFA-eLearning**.

La plateforme intègre plusieurs composantes technologiques : un backend développé avec **Spring Boot** (Java), un frontend réalisé avec **React.js** et **Vite**, et un moteur de recommandation basé sur l'intelligence artificielle développé avec **Python** et des algorithmes de traitement du langage naturel (NLP).

Le système permet aux étudiants de s'inscrire à des cours, de suivre des exercices et des quiz, et de recevoir des recommandations personnalisées basées sur leur historique d'apprentissage. Les enseignants peuvent créer et gérer des cours, des exercices et des quiz via une interface dédiée.

Le projet a été conduit selon la méthodologie agile **Scrum**, structuré en quatre sprints couvrant respectivement l'authentification et la gestion des utilisateurs, la gestion des cours, les fonctionnalités pédagogiques (quiz et exercices), et le moteur de recommandation IA.

Mots-clés : E-Learning, Spring Boot, React.js, Intelligence Artificielle, Recommandation, NLP, Scrum, JWT, REST API.

Abstract

This report presents the work carried out as part of the Final Year Project (PFA), focusing on the design and development of an intelligent e-learning platform called **PFA-eLearning**.

The platform integrates several technological components : a backend developed with **Spring Boot** (Java), a frontend built with **React.js** and **Vite**, and an AI-powered recommendation engine developed with **Python** and Natural Language Processing (NLP) algorithms.

The project was conducted following the agile **Scrum** methodology, structured into four sprints covering authentication, course management, educational features (quizzes and exercises), and the AI recommendation engine.

Keywords : E-Learning, Spring Boot, React.js, Artificial Intelligence, Recommendation, NLP, Scrum, JWT, REST API.

Table des matières

Dédicaces i

Remerciements ii

Résumé iii

Liste des figures viii

Liste des tableaux ix

Introduction Générale 1

1 Étude Préalable et Spécification des Besoins 3
- 1.1 Présentation du projet . . . 3
  - 1.1.1 Contexte et motivation . . . 3
  - 1.1.2 Périmètre du projet . . . 3
- 1.2 Étude de l'existant . . . 3
  - 1.2.1 Analyse des plateformes existantes . . . 4
  - 1.2.2 Critique de l'existant . . . 4
- 1.3 Spécification des besoins . . . 4
  - 1.3.1 Besoins fonctionnels . . . 4
  - 1.3.2 Besoins non fonctionnels . . . 5
- 1.4 Méthodologie adoptée : Scrum . . . 5
  - 1.4.1 Présentation de Scrum . . . 5
  - 1.4.2 Rôles Scrum dans notre équipe . . . 6
  - 1.4.3 Cérémonies Scrum . . . 6
  - 1.4.4 Product Backlog . . . 6

2 Conception Architecturale du Système 8
- 2.1 Architecture globale . . . 8
  - 2.1.1 Vue d'ensemble . . . 8
  - 2.1.2 Diagramme d'architecture . . . 9
- 2.2 Architecture du Backend (Spring Boot) . . . 9

2.2.1 Structure en couches 9
2.2.2 Modèle de données 9
2.2.3 Sécurité – Architecture JWT 10
2.3 Architecture du Frontend (React.js) 10
2.3.1 Structure de l'application 10
2.3.2 Gestion de l'état et authentification 11
2.4 Architecture du Moteur IA (Python) 11
2.4.1 Structure du service 11
2.4.2 Algorithme de recommandation 11

3 Sprints 1 et 2 – Authentification et Gestion des Cours 12
3.1 Sprint 1 – Authentification et Gestion des Utilisateurs 12
3.1.1 Sprint Backlog – Sprint 1 13
3.1.2 Réalisations techniques – Sprint 1 13
3.1.3 Sprint Review – Sprint 1 15
3.1.4 Sprint Retrospective – Sprint 1 15
3.2 Sprint 2 – Gestion des Cours et Tableaux de Bord 16
3.2.1 Sprint Backlog – Sprint 2 16
3.2.2 Réalisations techniques – Sprint 2 16
3.2.3 Sprint Review – Sprint 2 17
3.2.4 Sprint Retrospective – Sprint 2 17

4 Sprint 3 – Fonctionnalités Pédagogiques 18
4.1 Sprint Backlog – Sprint 3 18
4.2 Modèle du Quiz 18
4.2.1 Architecture du module Quiz 19
4.2.2 Algorithme de correction 19
4.3 Module Exercices 20
4.3.1 Architecture du module Exercice 20
4.4 Interface Utilisateur – Quiz et Exercices 20
4.4.1 Page QuizPage 20
4.4.2 Page ExercisesPage 20
4.5 Résultats du Sprint 3 21
4.5.1 Sprint Review 21
4.5.2 Sprint Retrospective 21

5 Sprint 4 – Moteur de Recommandation IA 22
5.1 Architecture du Service IA 22
5.1.1 Vue d'ensemble 22

5.1.2 Sprint Backlog – Sprint 4 23
5.2 Implémentation du Moteur NLP 23
5.2.1 Extracteur de texte (TextExtractor) 23
5.2.2 Algorithme de recommandation hybride 24
5.3 Intégration Backend – Service IA 26
5.3.1 RecommendationService (Spring Boot) 26
5.3.2 Module de recherche intelligente 26
5.4 Interface Utilisateur – Recommandations 26
5.4.1 Intégration dans le tableau de bord étudiant 26
5.4.2 Page de recherche (SearchPage) 26
5.5 Sprint Review – Sprint 4 27
5.6 Sprint Retrospective – Sprint 4 27

Conclusion Générale et Perspectives **28**

Références Bibliographiques **31**

A Annexe A – Configuration application.yml **33**

B Annexe B – Requirements Python (Service IA) **35**

C Annexe C – Tableau récapitulatif des endpoints REST **36**

Table des figures

2.1 Architecture globale du système PFA-eLearning 9

Liste des tableaux

1.1 Comparaison des plateformes e-learning existantes 4
1.2 Besoins non fonctionnels du système 5
1.3 Répartition des rôles Scrum 6
1.4 Product Backlog – Extrait des User Stories prioritaires 7

2.1 Entités principales du modèle de données 10

3.1 Sprint Backlog – Sprint 1 13
3.2 Sprint Backlog – Sprint 2 16

4.1 Sprint Backlog – Sprint 3 18

5.1 Sprint Backlog – Sprint 4 23

C.1 Principaux endpoints REST du backend 36

Introduction Générale

Contexte général

L'ère numérique a profondément transformé le secteur de l'éducation. L'apprentissage en ligne (e-learning) est devenu une réalité incontournable, offrant aux apprenants une flexibilité et une accessibilité sans précédent. Face à la diversité des contenus disponibles, il devient cependant difficile pour un étudiant de trouver les ressources les plus adaptées à ses besoins spécifiques, son niveau et ses objectifs d'apprentissage.

C'est dans ce contexte que s'inscrit notre Projet de Fin d'Année : développer une plateforme e-learning intelligente capable non seulement de centraliser les ressources pédagogiques, mais aussi de guider chaque apprenant vers les contenus les plus pertinents grâce à un moteur de recommandation basé sur l'intelligence artificielle.

Problématique

La problématique principale que nous avons identifiée est la suivante : comment concevoir et développer une plateforme e-learning complète et moderne qui offre une expérience d'apprentissage personnalisée, intuitive et efficace, tout en intégrant des fonctionnalités avancées de recommandation intelligente ?

Objectifs du projet

Les objectifs principaux de notre projet sont :

- Concevoir une architecture logicielle robuste et scalable suivant les meilleures pratiques du génie logiciel.
- Développer un système d'authentification sécurisé basé sur les tokens JWT.
- Créer un espace dédié aux enseignants pour la gestion des cours, exercices et quiz.
- Proposer aux étudiants une interface intuitive pour l'apprentissage et le suivi de leur progression.
- Intégrer un moteur de recommandation IA pour personnaliser l'expérience d'apprentissage.
- Appliquer la méthodologie Scrum pour une gestion de projet agile et efficace.

Structure du rapport

Ce rapport est organisé en cinq chapitres :

- **Chapitre 1 :** Étude préalable et spécification des besoins.
- **Chapitre 2 :** Conception architecturale du système.
- **Chapitre 3 :** Sprint 1 et Sprint 2 – Authentification et gestion des cours.
- **Chapitre 4 :** Sprint 3 – Fonctionnalités pédagogiques (Quiz et Exercices).
- **Chapitre 5 :** Sprint 4 – Moteur de recommandation IA.

Chapitre 1

Étude Préalable et Spécification des Besoins

1.1 Présentation du projet

1.1.1 Contexte et motivation

La démocratisation d'Internet et la montée en puissance des technologies mobiles ont créé un terrain fertile pour le développement de solutions éducatives numériques. Les plateformes e-learning comme Coursera, Udemy ou Moodle ont démontré l'immense potentiel de ce marché. Toutefois, la plupart de ces plateformes manquent de personnalisation réelle et ne s'adaptent pas dynamiquement aux besoins individuels des apprenants.

Notre projet **PFA-eLearning** vise à combler ce manque en proposant une solution complète, depuis la gestion des utilisateurs jusqu'à la recommandation intelligente de contenus pédagogiques.

1.1.2 Périmètre du projet

La plateforme PFA-eLearning s'adresse à trois types d'utilisateurs :

— **Administrateurs** : gestion globale de la plateforme, des utilisateurs et des catégories.

— **Enseignants** : création et gestion de cours, d'exercices et de quiz.

— **Étudiants** : inscription aux cours, suivi de la progression, participation aux quiz et accès aux recommandations personnalisées.

1.2 Étude de l'existant

1.2.1 Analyse des plateformes existantes

Avant de commencer le développement, nous avons analysé plusieurs plateformes e-learning existantes afin d'identifier leurs forces et leurs limites.

Table 1.1 – Comparaison des plateformes e-learning existantes

Critère	**Moodle**	**Udemy**	**PFA-eLearning**
Open Source	Oui	Non	Oui
Recommandation IA	Non	Partielle	Oui (NLP)
API REST	Limitée	Oui	Oui
Gestion des quiz	Oui	Limitée	Oui
Architecture moderne	Non	Oui	Oui
Personnalisation	Faible	Moyenne	Élevée

1.2.2 Critique de l'existant

Les analyses réalisées révèlent plusieurs lacunes dans les solutions existantes :

- Architecture monolithique difficile à maintenir et à faire évoluer.
- Manque de personnalisation basée sur l'IA.
- Interfaces utilisateurs vieillissantes et peu réactives.
- Absence d'APIs RESTful modernes permettant l'intégration avec des services tiers.

Notre solution apporte des réponses concrètes à ces problèmes grâce à une architecture microservices, un frontend réactif et un moteur de recommandation IA.

1.3 Spécification des besoins

1.3.1 Besoins fonctionnels

Module Authentification

- Inscription d'un nouvel utilisateur (étudiant ou enseignant).
- Connexion avec génération d'un token JWT.
- Gestion des rôles (ADMIN, TEACHER, STUDENT).
- Déconnexion et invalidation du token.

Module Cours

- Création, modification et suppression de cours par les enseignants.
- Téléchargement de fichiers multimédia (vidéos, PDF, images).

— Catégorisation et classification par niveau de difficulté.
— Inscription des étudiants aux cours.
— Notation et évaluation des cours (système de rating).

Module Quiz et Exercices

— Création de quiz avec questions à choix multiples.
— Soumission de réponses et calcul automatique du score.
— Historique des résultats et statistiques de performance.
— Création d'exercices pratiques avec correction.
— Suivi de la complétion des exercices.

Module Recommandation

— Analyse du profil d'apprentissage de l'étudiant.
— Extraction de texte et analyse NLP des cours.
— Génération de recommandations personnalisées.
— Recherche intelligente de cours.
— Historique des recherches et préférences.

1.3.2 Besoins non fonctionnels

Table 1.2 – Besoins non fonctionnels du système

Critère	**Exigence**
Performance	Temps de réponse API inférieur à 500 ms pour 95% des requêtes
Sécurité	Authentification JWT, chiffrement des mots de passe (BCrypt)
Scalabilité	Architecture permettant l'ajout de nouveaux modules
Disponibilité	Uptime cible de 99%
Utilisabilité	Interface responsive (desktop et mobile)
Maintenabilité	Code documenté, tests unitaires, architecture en couches

1.4 Méthodologie adoptée : Scrum

1.4.1 Présentation de Scrum

Pour la gestion de ce projet, nous avons adopté la méthodologie agile **Scrum**. Cette méthode itérative et incrémentielle est particulièrement adaptée aux projets de développement logiciel où les exigences peuvent évoluer au cours du temps.

Scrum repose sur trois piliers fondamentaux :

- **Transparence** : Toute l'équipe a une vision claire de l'avancement.
- **Inspection** : Des points réguliers permettent d'évaluer la progression.
- **Adaptation** : L'équipe s'ajuste en fonction des retours obtenus.

1.4.2 Rôles Scrum dans notre équipe

Table 1.3 – Répartition des rôles Scrum

Rôle	Membre	Responsabilités principales
Product Owner	BAHLOUL Donia	Gestion du backlog, priorisation
Scrum Master	FRIKHA Amin	Animation des cérémonies, levée des obstacles
Dev Team	ABDELHEDI Wassim	Développement, tests, livraison
Dev Team	BAHLOUL Donia	Frontend, UI/UX
Dev Team	FRIKHA Amin	Backend, API REST

1.4.3 Cérémonies Scrum

Les cérémonies Scrum suivantes ont été appliquées tout au long du projet :

- **Sprint Planning** : Planification des tâches au début de chaque sprint (durée : 2 semaines).
- **Daily Scrum** : Réunion quotidienne de 15 minutes pour synchroniser l'équipe.
- **Sprint Review** : Démonstration du travail accompli en fin de sprint.
- **Sprint Retrospective** : Analyse des points positifs et axes d'amélioration.

1.4.4 Product Backlog

Le Product Backlog regroupe l'ensemble des User Stories identifiées pour le projet, classées par ordre de priorité.

Table 1.4 – Product Backlog – Extrait des User Stories prioritaires

ID	En tant que	Je veux	Priorité	Sprint
US-01	Étudiant	M'inscrire sur la plateforme	Haute	S1
US-02	Utilisateur	Me connecter via JWT	Haute	S1
US-03	Admin	Gérer les utilisateurs	Haute	S1
US-04	Enseignant	Créer un cours	Haute	S2
US-05	Étudiant	M'inscrire à un cours	Haute	S2
US-06	Enseignant	Créer un quiz	Moyenne	S3
US-07	Étudiant	Passer un quiz	Moyenne	S3
US-08	Enseignant	Créer un exercice	Moyenne	S3
US-09	Étudiant	Rechercher des cours	Haute	S4
US-10	Étudiant	Recevoir des recommandations	Haute	S4
US-11	Étudiant	Voir mon tableau de bord	Moyenne	S2
US-12	Enseignant	Voir mon tableau de bord	Moyenne	S2

Chapitre 2

Conception Architecturale du Système

2.1 Architecture globale

2.1.1 Vue d'ensemble

L'architecture de PFA-eLearning suit une approche **orientée microservices** avec une séparation claire entre le frontend, le backend principal et le service de recommandation IA. Cette approche garantit la modularité, la maintenabilité et la scalabilité du système.

Le système est composé de trois composantes principales :

1. **elearning-frontend** : Application Single Page Application (SPA) développée avec React.js.
2. **elearning-backend** : API RESTful développée avec Spring Boot (Java).
3. **ai-recommendation-engine** : Service Python dédié aux algorithmes de recommandation et de traitement NLP.

2.1.2 Diagramme d'architecture

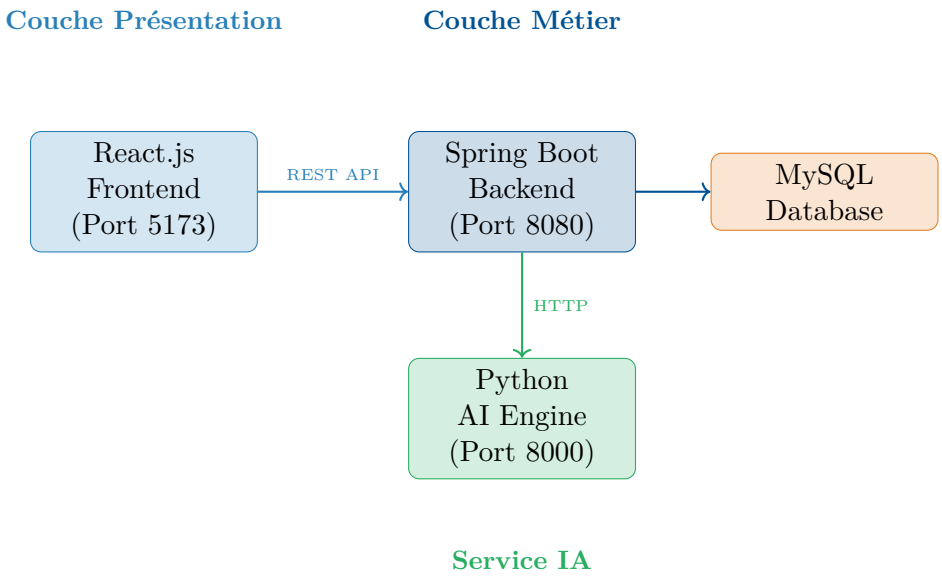

FIGURE 2.1 – Architecture globale du système PFA-eLearning

2.2 Architecture du Backend (Spring Boot)

2.2.1 Structure en couches

Le backend suit une architecture en couches (*layered architecture*) conforme aux meilleures pratiques de Spring Boot :

- **Controller** : Exposition des endpoints REST, validation des entrées.
- **Service** : Logique métier, orchestration des opérations.
- **Repository** : Accès aux données via Spring Data JPA.
- **Model** : Entités JPA mappées sur la base de données.
- **DTO** : Objets de transfert de données (Request/Response).
- **Security** : Filtres JWT, gestion des rôles et des autorisations.

2.2.2 Modèle de données

Le modèle de données du système comprend les entités principales suivantes :

Table 2.1 – Entités principales du modèle de données

Entité	Description
User	Utilisateur du système avec rôle (ADMIN/TEACHER/STUDENT)
Course	Cours avec titre, description, niveau de difficulté
Category	Catégorie de classification des cours
Enrollment	Inscription d'un étudiant à un cours
Quiz	Quiz associé à un cours
QuizQuestion	Question d'un quiz avec choix multiples
QuizResult	Résultat d'un étudiant à un quiz
Exercise	Exercice pratique associé à un cours
ExerciseCompletion	Suivi de la complétion d'un exercice
Recommendation	Recommandation générée pour un étudiant
SearchHistory	Historique des recherches d'un utilisateur
CourseRating	Note attribuée à un cours par un étudiant

2.2.3 Sécurité – Architecture JWT

La sécurité du système repose sur l'utilisation de **JSON Web Tokens (JWT)**. Le flux d'authentification se déroule comme suit :

1. L'utilisateur envoie ses identifiants (email + mot de passe) au endpoint `/api/auth/login`.
2. Le backend vérifie les identifiants via `CustomUserDetailsService`.
3. En cas de succès, `JwtTokenProvider` génère un token signé.
4. Le token est retourné au client dans la réponse `AuthResponse`.
5. Pour chaque requête suivante, le client inclut le token dans le header `Authorization: Bearer <token>`.
6. `JwtAuthenticationFilter` intercepte et valide le token à chaque requête.

2.3 Architecture du Frontend (React.js)

2.3.1 Structure de l'application

Le frontend est développé avec React.js et Vite, suivant une architecture modulaire :

— **/api** : Couche d'abstraction pour les appels API (axios).
— **/components** : Composants réutilisables (Navbar, CourseCard, SearchBar).
— **/context** : Gestion de l'état global (AuthContext avec React Context API).
— **/pages** : Pages de l'application organisées par rôle (auth, student, teacher).

2.3.2 Gestion de l'état et authentification

L'état d'authentification est géré via **React Context API** (`AuthContext`), qui centralise les informations sur l'utilisateur connecté et expose les fonctions de connexion/-déconnexion à travers l'arbre de composants.

La protection des routes est assurée par le composant `ProtectedRoute`, qui vérifie le rôle de l'utilisateur avant d'autoriser l'accès à une page.

2.4 Architecture du Moteur IA (Python)

2.4.1 Structure du service

Le moteur de recommandation est un service Python indépendant, structuré comme suit :

- **models/recommender.py** : Algorithme de recommandation collaborative et basé sur le contenu.
- **models/nlp_processor.py** : Traitement du langage naturel pour l'analyse des contenus.
- **services/text_extractor.py** : Extraction et nettoyage du texte des cours.
- **routes/recommendation_routes.py** : Endpoints REST du service IA.
- **schemas/recommendation_schema.py** : Schémas de validation Pydantic.

2.4.2 Algorithme de recommandation

Le moteur de recommandation combine deux approches complémentaires :

1. **Filtrage basé sur le contenu (Content-Based Filtering)** : Analyse les métadonnées des cours (titre, description, catégorie) et du profil de l'étudiant pour identifier des cours similaires à ceux déjà suivis.
2. **Filtrage collaboratif (Collaborative Filtering)** : Identifie les patterns comportementaux entre étudiants ayant des profils similaires pour proposer des cours populaires dans leur groupe.

Le NLP est utilisé pour extraire des *features* sémantiques des descriptions de cours, calculer des similarités cosinus entre vecteurs TF-IDF, et analyser les requêtes de recherche.

Chapitre 3

Sprints 1 et 2 – Authentification et Gestion des Cours

3.1 Sprint 1 – Authentification et Gestion des Utilisateurs

Sprint 1 – Authentification et Gestion des Utilisateurs

Durée : 2 semaines **Responsable principal :** FRIKHA Amin (Backend) & BAHLOUL Donia (Frontend)

Objectif : Mettre en place le système d'authentification sécurisé et la gestion des utilisateurs.

3.1.1 Sprint Backlog – Sprint 1

Table 3.1 – Sprint Backlog – Sprint 1

ID	Tâche	Assigné à	Effort (h)
T-01	Configuration Spring Boot + Spring Security	FRIKHA Amin	4
T-02	Implémentation JwtTokenProvider	FRIKHA Amin	6
T-03	Implémentation JwtAuthenticationFilter	FRIKHA Amin	4
T-04	Entité User + UserRepository	ABDELHEDI Wassim	3
T-05	Service d'inscription (RegisterRequest)	ABDELHEDI Wassim	5
T-06	Service de connexion (LoginRequest/AuthResponse)	ABDELHEDI Wassim	5
T-07	AuthController (endpoints /login, /register)	FRIKHA Amin	4
T-08	AdminController – Gestion utilisateurs	ABDELHEDI Wassim	6
T-09	Page LoginPage (React)	BAHLOUL Donia	5
T-10	Page RegisterPage (React)	BAHLOUL Donia	5
T-11	AuthContext + ProtectedRoute	BAHLOUL Donia	6
T-12	Configuration Axios (axiosConfig.js)	BAHLOUL Donia	3

3.1.2 Réalisations techniques – Sprint 1

Backend : Sécurité JWT

La classe `JwtTokenProvider` est au cœur du système de sécurité. Elle assure la génération, la validation et l'extraction des informations contenues dans le token JWT.

Listing 3.1 – Extrait de JwtTokenProvider.java

```java
@Component
public class JwtTokenProvider {

    @Value("${jwt.secret}")
    private String jwtSecret;

    @Value("${jwt.expiration}")
    private int jwtExpiration;

    public String generateToken(Authentication authentication)
        {
         UserDetails userDetails =
```

```
            (UserDetails) authentication.getPrincipal();
        return Jwts.builder()
            .setSubject(userDetails.getUsername())
            .setIssuedAt(new Date())
            .setExpiration(new Date(new Date().getTime()
                + jwtExpiration))
            .signWith(SignatureAlgorithm.HS512, jwtSecret)
            .compact();
    }

    public boolean validateToken(String token) {
        Jwts.parser().setSigningKey(jwtSecret)
            .parseClaimsJws(token);
        return true;
    }
}
```

Frontend : AuthContext

Le contexte d'authentification centralise la gestion de l'état d'authentification dans l'application React.

Listing 3.2 – Extrait de AuthContext.jsx

```
export const AuthProvider = ({ children }) => {
  const [user, setUser] = useState(null);
  const [token, setToken] = useState(
    localStorage.getItem('token') || null
  );

  const login = async (credentials) => {
    const response = await authApi.login(credentials);
    setToken(response.data.token);
    setUser(response.data.user);
    localStorage.setItem('token', response.data.token);
  };

  const logout = () => {
    setUser(null);
    setToken(null);
    localStorage.removeItem('token');
  };
```

```

  return (
    <AuthContext.Provider value={{ user, token, login, logout }}>
      {children}
    </AuthContext.Provider>
  );
};
```

3.1.3 Sprint Review – Sprint 1

À l'issue du Sprint 1, les fonctionnalités suivantes ont été livrées et validées :

- ✓ Système d'inscription avec validation des données.
- ✓ Authentification par token JWT avec expiration configurable.
- ✓ Gestion des rôles (ADMIN, TEACHER, STUDENT).
- ✓ Interface de connexion et d'inscription responsive.
- ✓ Protection des routes côté frontend.
- ✓ Panel administrateur pour la gestion des utilisateurs.

3.1.4 Sprint Retrospective – Sprint 1

Points positifs :

— Bonne répartition des tâches entre les membres.

— La documentation Spring Security a facilité l'implémentation.

Points d'amélioration :

— Prévoir plus de temps pour les tests d'intégration.

— Améliorer la gestion des messages d'erreur côté frontend.

3.2 Sprint 2 – Gestion des Cours et Tableaux de Bord

Sprint 2 – Gestion des Cours

Durée : 2 semaines **Responsable principal :** ABDELHEDI Wassim (Backend) & BAHLOUL Donia (Frontend)

Objectif : Développer les fonctionnalités de gestion des cours, d'inscription et les tableaux de bord.

3.2.1 Sprint Backlog – Sprint 2

Table 3.2 – Sprint Backlog – Sprint 2

ID	Tâche	Assigné à	Effort (h)
T-13	Entité Course + CourseRepository	ABDELHEDI Wassim	4
T-14	CourseService (CRUD complet)	ABDELHEDI Wassim	8
T-15	CourseController (REST endpoints)	FRIKHA Amin	6
T-16	FileStorageService (upload fichiers)	FRIKHA Amin	8
T-17	Entités Category + CategoryRepository	ABDELHEDI Wassim	3
T-18	CategoryController + Service	FRIKHA Amin	4
T-19	Système d'inscription aux cours	ABDELHEDI Wassim	6
T-20	Système de notation des cours	FRIKHA Amin	5
T-21	Page CoursesPage (liste des cours)	BAHLOUL Donia	6
T-22	Page CourseViewPage (détail d'un cours)	BAHLOUL Donia	7
T-23	Page CreateCourse (enseignant)	BAHLOUL Donia	6
T-24	Dashboard étudiant	BAHLOUL Donia	5
T-25	TeacherDashboard	BAHLOUL Donia	5

3.2.2 Réalisations techniques – Sprint 2

Backend : Gestion des cours

Le `CourseService` implémente la logique métier complète pour la gestion des cours, incluant la création, la mise à jour, la suppression et la récupération des cours avec pagination et filtrage.

La gestion des fichiers (upload de supports de cours) est assurée par le `FileStorageService` qui :

- Valide les types de fichiers acceptés (PDF, vidéo, image).
- Génère des noms de fichiers uniques pour éviter les conflits.

— Stocke les fichiers dans un répertoire configuré.
— Expose les fichiers via le `WebConfig`.

Frontend : CourseCard et CoursesPage

Le composant `CourseCard` offre une présentation visuelle attrayante de chaque cours, incluant l'image de couverture, le titre, la description résumée, la catégorie, le niveau de difficulté et la note moyenne.

La page `CoursesPage` intègre un système de filtrage dynamique par catégorie et niveau de difficulté, permettant à l'étudiant de trouver rapidement les cours qui correspondent à ses besoins.

3.2.3 Sprint Review – Sprint 2

Fonctionnalités livrées et validées à l'issue du Sprint 2 :

✓ CRUD complet pour les cours (création, lecture, modification, suppression).
✓ Upload et stockage des fichiers multimédia.
✓ Système de catégories et niveaux de difficulté.
✓ Inscription des étudiants aux cours.
✓ Système de notation des cours.
✓ Tableaux de bord étudiant et enseignant.
✓ Interface de création de cours pour les enseignants.

3.2.4 Sprint Retrospective – Sprint 2

Points positifs :

— L'utilisation de Spring Data JPA a significativement accéléré le développement.
— La composantisation React a permis une bonne réutilisabilité du code.

Points d'amélioration :

— Optimiser les requêtes de base de données (ajout d'index).
— Améliorer la gestion des erreurs de téléchargement de fichiers.

Chapitre 4

Sprint 3 – Fonctionnalités Pédagogiques

Sprint 3 – Quiz et Exercices

Durée : 2 semaines **Responsable principal :** FRIKHA Amin (Quiz Backend) & ABDELHEDI Wassim (Exercices)

Objectif : Développer les modules de quiz interactifs et d'exercices pratiques avec suivi de progression.

4.1 Sprint Backlog – Sprint 3

Table 4.1 – Sprint Backlog – Sprint 3

ID	Tâche	Assigné à	Effort (h)
T-26	Entités Quiz, QuizQuestion, QuizResult	FRIKHA Amin	5
T-27	QuizRepository + QuizResultRepository	FRIKHA Amin	3
T-28	Logique de correction automatique des quiz	FRIKHA Amin	8
T-29	QuizController (REST endpoints)	FRIKHA Amin	5
T-30	Entités Exercise + ExerciseCompletion	ABDELHEDI Wassim	4
T-31	ExerciseService + ExerciseController	ABDELHEDI Wassim	7
T-32	Suivi de complétion des exercices	ABDELHEDI Wassim	5
T-33	Page QuizPage (React)	BAHLOUL Donia	8
T-34	Page ExercisesPage (React)	BAHLOUL Donia	6
T-35	Page CreateQuiz (enseignant)	BAHLOUL Donia	7
T-36	Page CreateExercise (enseignant)	BAHLOUL Donia	6
T-37	Statistiques de performance	ABDELHEDI Wassim	5

4.2 Modèle du Quiz

4.2.1 Architecture du module Quiz

Le module Quiz est composé de trois entités principales liées entre elles :

- **Quiz** : Entité principale contenant le titre, la description et le seuil de réussite. Un quiz est associé à un cours spécifique.
- **QuizQuestion** : Chaque question contient un énoncé, quatre options de réponse (optionA, optionB, optionC, optionD) et l'indication de la bonne réponse.
- **QuizResult** : Enregistre le score obtenu, les réponses fournies et la date de passage pour chaque tentative d'un étudiant.

4.2.2 Algorithme de correction

La correction automatique des quiz est implémentée dans le `QuizService`. Pour chaque question, la réponse de l'étudiant est comparée à la réponse correcte. Le score final est calculé en pourcentage et comparé au seuil de réussite défini par l'enseignant.

Listing 4.1 – Algorithme de correction des quiz

```java
public QuizResult submitQuiz(Long quizId, Long studentId,
                             Map<Long, String> answers) {
    Quiz quiz = quizRepository.findById(quizId)
        .orElseThrow(() -> new ResourceNotFoundException("Quiz"
           ));

    int correctAnswers = 0;
    int totalQuestions = quiz.getQuestions().size();

    for (QuizQuestion question : quiz.getQuestions()) {
        String studentAnswer = answers.get(question.getId());
        if (question.getCorrectAnswer().equals(studentAnswer))
           {
            correctAnswers++;
        }
    }

    double score = (correctAnswers * 100.0) / totalQuestions;
    boolean passed = score >= quiz.getPassingScore();

    QuizResult result = new QuizResult();
    result.setScore(score);
    result.setPassed(passed);
```

```
    result.setStudent(studentRepository.findById(studentId).get
        ());
    result.setQuiz(quiz);
    result.setCompletedAt(LocalDateTime.now());

    return quizResultRepository.save(result);
}
```

4.3 Module Exercices

4.3.1 Architecture du module Exercice

Le module exercice permet aux enseignants de créer des exercices pratiques associés à leurs cours. Chaque exercice contient :

- Un titre et une description de l'exercice.
- Des instructions détaillées pour l'étudiant.
- Un fichier de ressources optionnel.
- Un critère de validation (description de la solution attendue).

Le suivi de complétion est assuré par l'entité `ExerciseCompletion` qui enregistre la date de complétion et d'éventuels commentaires de l'étudiant.

4.4 Interface Utilisateur – Quiz et Exercices

4.4.1 Page QuizPage

La page `QuizPage` offre une expérience interactive à l'étudiant :

- Présentation une question à la fois avec indicateur de progression.
- Sélection visuelle des réponses avec feedback coloré.
- Timer optionnel pour les quiz chronométrés.
- Affichage du résultat final avec bilan détaillé.

4.4.2 Page ExercisesPage

La page `ExercisesPage` liste les exercices disponibles pour un cours donné, avec indication de l'état de complétion de chaque exercice (non commencé, en cours, terminé).

4.5 Résultats du Sprint 3

4.5.1 Sprint Review

Fonctionnalités livrées à l'issue du Sprint 3 :

- ✓ Création et gestion complète des quiz par les enseignants.
- ✓ Passage des quiz par les étudiants avec correction automatique.
- ✓ Historique des résultats et statistiques de performance.
- ✓ Création et gestion des exercices pratiques.
- ✓ Suivi de complétion des exercices.
- ✓ Interfaces React intuitives pour quiz et exercices.

4.5.2 Sprint Retrospective

Points positifs :

— L'algorithme de correction automatique est robuste et extensible.

— L'interface de quiz a été bien reçue lors des tests utilisateurs.

Points d'amélioration :

— Ajouter un système de reprise d'un quiz en cours de session.

— Permettre des questions de type vrai/faux et texte libre.

Chapitre 5

Sprint 4 – Moteur de Recommandation IA

Sprint 4 – Intelligence Artificielle et Recommandation

Durée : 2 semaines **Responsable principal :** ABDELHEDI Wassim (AI Engine) & FRIKHA Amin (Intégration)

Objectif : Développer et intégrer le moteur de recommandation IA basé sur le NLP pour personnaliser l'expérience d'apprentissage.

5.1 Architecture du Service IA

5.1.1 Vue d'ensemble

Le service de recommandation IA est développé en **Python** avec le framework **FastAPI**. Il fonctionne comme un microservice indépendant exposant des endpoints REST consommés par le backend Spring Boot.

5.1.2 Sprint Backlog – Sprint 4

Table 5.1 – Sprint Backlog – Sprint 4

ID	Tâche	Assigné à	Effort (h)
T-38	Setup FastAPI + configuration	ABDELHEDI Wassim	3
T-39	Implémentation TextExtractor	ABDELHEDI Wassim	6
T-40	Implémentation NLPProcessor (TF-IDF)	ABDELHEDI Wassim	10
T-41	Algorithme de recommandation hybride	ABDELHEDI Wassim	12
T-42	Endpoints REST du service IA	FRIKHA Amin	5
T-43	Schémas Pydantic (validation)	FRIKHA Amin	3
T-44	RecommendationService (Spring Boot)	FRIKHA Amin	6
T-45	RecommendationController (Spring Boot)	FRIKHA Amin	4
T-46	SearchService + SearchController	FRIKHA Amin	6
T-47	Historique des recherches	ABDELHEDI Wassim	4
T-48	Page SearchPage (React)	BAHLOUL Donia	7
T-49	Intégration recommandations dans Dashboard	BAHLOUL Donia	5

5.2 Implémentation du Moteur NLP

5.2.1 Extracteur de texte (TextExtractor)

Le module `text_extractor.py` assure le prétraitement des données textuelles extraites des cours. Les étapes de traitement incluent :

1. **Nettoyage du texte** : Suppression des caractères spéciaux, ponctuation, chiffres.
2. **Normalisation** : Conversion en minuscules, suppression des espaces multiples.
3. **Tokenisation** : Découpage du texte en tokens (mots).
4. **Suppression des stop words** : Élimination des mots courants non significatifs (le, la, de, etc.).
5. **Lemmatisation** : Réduction des mots à leur forme canonique.

Listing 5.1 – Extrait de nlp$_{p}rocessor.py$

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
import numpy as np

```

```python
class NLPProcessor:
    def __init__(self):
        self.vectorizer = TfidfVectorizer(
            max_features=5000,
            stop_words='english',
            ngram_range=(1, 2)
        )
        self.tfidf_matrix = None
        self.course_ids = []

    def fit(self, courses: list):
        """Entra ne le vectoriseur sur les descriptions de cours."""
        texts = [c['title'] + ' ' + c['description']
                 for c in courses]
        self.tfidf_matrix = self.vectorizer.fit_transform(texts)
        self.course_ids = [c['id'] for c in courses]

    def get_similar_courses(self, course_id: int, top_n: int = 5):
        """Retourne les N cours les plus similaires."""
        idx = self.course_ids.index(course_id)
        similarities = cosine_similarity(
            self.tfidf_matrix[idx], self.tfidf_matrix
        ).flatten()
        similar_indices = similarities.argsort()[::-1][1:top_n+1]
        return [self.course_ids[i] for i in similar_indices]
```

5.2.2 Algorithme de recommandation hybride

Le `recommender.py` implémente l'algorithme hybride combinant les deux approches :

Listing 5.2 – Extrait de recommender.py

```python
class HybridRecommender:
    def __init__(self, nlp_processor: NLPProcessor):
        self.nlp = nlp_processor

    def recommend(self, student_profile: dict,
                  all_courses: list, top_n: int = 10):
```

```
        """
        G n re des recommandations personnalis es.

        Args:
            student_profile: Historique d'apprentissage
            all_courses: Tous les cours disponibles
            top_n: Nombre de recommandations  retourner
        """
        enrolled_ids = student_profile.get('enrolled_courses', []) 
        category_prefs = student_profile.get('categories', [])

        scores = {}
        for course in all_courses:
            if course['id'] in enrolled_ids:
                continue

            score = 0.0
            # Score bas  sur le contenu
            for enrolled_id in enrolled_ids:
                similar = self.nlp.get_similar_courses(enrolled_id)
                if course['id'] in similar:
                    score += 2.0

            # Score bas  sur les pr f rences de cat gories
            if course['category'] in category_prefs:
                score += 1.5

            # Score bas  sur la popularit
            score += course.get('enrollment_count', 0) * 0.01

            scores[course['id']] = score

        sorted_courses = sorted(
            scores.items(), key=lambda x: x[1], reverse=True
        )
        return [c[0] for c in sorted_courses[:top_n]]
```

5.3 Intégration Backend – Service IA

5.3.1 RecommendationService (Spring Boot)

Le `RecommendationService` côté Spring Boot joue le rôle de pont entre l'application principale et le microservice IA. Il :

- Collecte le profil d'apprentissage de l'étudiant (cours suivis, catégories préférées).
- Appelle l'API du service Python via `RestTemplate` ou `WebClient`.
- Récupère les identifiants des cours recommandés.
- Hydrate les données complètes des cours depuis la base de données.
- Persiste les recommandations générées dans la table `Recommendation`.

5.3.2 Module de recherche intelligente

Le module de recherche, implémenté dans `SearchService`, offre une recherche avancée :

- Recherche full-text sur le titre, la description et les catégories des cours.
- Historisation automatique des requêtes de recherche (`SearchHistory`).
- Suggestions de recherche basées sur l'historique.
- Résultats triés par pertinence (score de correspondance).

5.4 Interface Utilisateur – Recommandations

5.4.1 Intégration dans le tableau de bord étudiant

Les recommandations sont affichées directement dans le tableau de bord de l'étudiant, dans une section dédiée « *Recommandé pour vous* ». Les cours recommandés sont présentés sous forme de cartes, identiques au composant `CourseCard`, mais avec un badge indiquant la raison de la recommandation (« Similaire à vos cours », « Populaire dans votre domaine », etc.).

5.4.2 Page de recherche (SearchPage)

La page `SearchPage` intègre la `SearchBar` et affiche les résultats en temps réel grâce à un mécanisme de debounce. Les résultats incluent des filtres par catégorie, niveau et durée, permettant d'affiner rapidement la recherche.

5.5 Sprint Review – Sprint 4

Fonctionnalités livrées à l'issue du Sprint 4 :

- ✓ Moteur NLP avec TF-IDF et similarité cosinus.
- ✓ Algorithme de recommandation hybride (contenu + collaboratif).
- ✓ Service Python FastAPI indépendant.
- ✓ Intégration complète backend Spring Boot ↔ service IA.
- ✓ Système de recherche intelligente avec historisation.
- ✓ Affichage des recommandations dans le tableau de bord.
- ✓ Page de recherche avec filtres dynamiques.

5.6 Sprint Retrospective – Sprint 4

Points positifs :

— L'architecture microservices s'est révélée très adaptée pour isoler la logique IA.

— Les algorithmes NLP ont produit des recommandations pertinentes lors des tests.

Points d'amélioration :

— Envisager l'utilisation de modèles de langage plus avancés (BERT, word2vec).

— Ajouter un mécanisme de feedback utilisateur pour améliorer les recommandations.

Conclusion Générale et Perspectives

Bilan du projet

Au terme de ce Projet de Fin d'Année, nous avons conçu et développé une plateforme e-learning intelligente et complète répondant aux besoins identifiés lors de l'analyse de l'existant. Le projet **PFA-eLearning** constitue une solution moderne et robuste qui intègre harmonieusement les dernières technologies du développement web et de l'intelligence artificielle.

Les quatre sprints Scrum ont permis de livrer de façon incrémentielle et structurée l'ensemble des fonctionnalités planifiées :

- Un système d'authentification sécurisé basé sur JWT (Sprint 1).
- Une gestion complète des cours avec upload de fichiers (Sprint 2).
- Des modules interactifs de quiz et d'exercices (Sprint 3).
- Un moteur de recommandation IA personnalisé (Sprint 4).

La méthodologie Scrum a joué un rôle fondamental dans le succès de ce projet. Les cérémonies régulières (daily scrum, reviews, rétrospectives) ont maintenu la cohésion de l'équipe et permis une adaptation continue aux imprévus et aux nouvelles exigences identifiées en cours de développement.

Compétences acquises

Ce projet nous a permis d'acquérir et de consolider de nombreuses compétences techniques et transversales :

Compétences techniques :

- Architecture microservices et conception d'API RESTful.
- Développement backend avec Spring Boot, Spring Security et JPA.
- Développement frontend avec React.js, hooks et Context API.
- Traitement du langage naturel (NLP) et algorithmes de recommandation.
- Gestion de la sécurité applicative (JWT, BCrypt, CORS).

Compétences transversales :

— Gestion de projet agile avec Scrum.
— Travail en équipe et communication technique.
— Documentation technique et rédaction de rapports.
— Résolution de problèmes complexes.

Perspectives et améliorations futures

Plusieurs pistes d'amélioration et d'extension peuvent être envisagées pour les versions futures de la plateforme :

1. **IA avancée** : Intégrer des modèles de langage plus sophistiqués (BERT, GPT) pour améliorer la qualité des recommandations et analyser les difficultés des étudiants.
2. **Apprentissage adaptatif** : Développer un système qui adapte dynamiquement le contenu et le rythme d'apprentissage en fonction des performances de chaque étudiant.
3. **Application mobile** : Développer des applications iOS et Android natives ou hybrides (React Native) pour étendre l'accessibilité de la plateforme.
4. **Fonctionnalités sociales** : Ajouter des forums de discussion, des espaces collaboratifs et des sessions d'apprentissage en groupe.
5. **Analytics avancée** : Implémenter un tableau de bord d'analytique pour les enseignants, offrant des statistiques détaillées sur l'engagement et la progression des étudiants.
6. **Gamification** : Introduire des mécanismes de gamification (badges, classements, points de progression) pour augmenter la motivation des apprenants.
7. **Déploiement Cloud** : Containeriser l'application avec Docker et orchestrer avec Kubernetes pour un déploiement scalable sur le cloud (AWS, Azure ou GCP).

Mot de la fin

Ce projet a représenté une expérience enrichissante à tous les niveaux. Nous avons eu l'opportunité d'appliquer concrètement les connaissances théoriques acquises tout au long de notre formation, de découvrir les défis réels du développement logiciel en équipe, et d'explorer des domaines innovants tels que l'intelligence artificielle appliquée à l'éducation.

Nous espérons que ce travail contribuera, même modestement, à l'avancement de l'éducation numérique et servira de base solide pour de futurs développements.

BAHLOUL Donia, FRIKHA Amin, ABDELHEDI Wassim

Sfax, 2025

Références Bibliographiques

Bibliographie

[1] Spring Framework Documentation, *Spring Boot Reference Guide – Version 3.x*, Disponible sur : https://docs.spring.io/spring-boot/docs/current/reference/html/, Consulté en 2024.

[2] Meta Open Source, *React – A JavaScript library for building user interfaces*, Disponible sur : https://react.dev/, Consulté en 2024.

[3] M. Jones, J. Bradley, N. Sakimura, *JSON Web Token (JWT) – RFC 7519*, IETF, Mai 2015.

[4] K. Schwaber, J. Sutherland, *The Scrum Guide – The Definitive Guide to Scrum*, Novembre 2020. Disponible sur : https://scrumguides.org/.

[5] G. Salton, C. Buckley, *Term-weighting approaches in automatic text retrieval*, Information Processing & Management, Vol. 24, No. 5, pp. 513-523, 1988.

[6] B. Sarwar, G. Karypis, J. Konstan, J. Riedl, *Item-based Collaborative Filtering Recommendation Algorithms*, Proceedings of the 10th WWW Conference, 2001.

[7] S. Ramírez, *FastAPI – Modern, fast web framework for building APIs with Python*, Disponible sur : https://fastapi.tiangolo.com/, Consulté en 2024.

[8] F. Pedregosa et al., *Scikit-learn : Machine Learning in Python*, Journal of Machine Learning Research, Vol. 12, pp. 2825-2830, 2011.

[9] Oracle Corporation, *MySQL 8.0 Reference Manual*, Disponible sur : https://dev.mysql.com/doc/, Consulté en 2024.

[10] E. You, *Vite – Next Generation Frontend Tooling*, Disponible sur : https://vitejs.dev/, Consulté en 2024.

Chapitre A

Annexe A – Configuration application.yml

Listing A.1 – Fichier de configuration Spring Boot (application.yml)

```yaml
spring:
  datasource:
    url: jdbc:mysql://localhost:3306/elearning_db
    username: root
    password: your_password
    driver-class-name: com.mysql.cj.jdbc.Driver
  jpa:
    hibernate:
      ddl-auto: update
    show-sql: true
    database-platform: org.hibernate.dialect.MySQLDialect
  servlet:
    multipart:
      enabled: true
      max-file-size: 100MB
      max-request-size: 100MB

server:
  port: 8080

jwt:
  secret: pfa-elearning-secret-key-2024
  expiration: 86400000

file:
  upload-dir: ./uploads

ai:
```

```
  recommendation:
    service-url: http://localhost:8000
```

Chapitre B

Annexe B – Requirements Python (Service IA)

Listing B.1 – requirements.txt du service IA

```
fastapi==0.104.1
uvicorn==0.24.0
scikit-learn==1.3.2
numpy==1.26.2
pandas==2.1.3
nltk==3.8.1
pydantic==2.5.1
httpx==0.25.2
python-multipart==0.0.6
```

Chapitre C

Annexe C – Tableau récapitulatif des endpoints REST

Table C.1 – Principaux endpoints REST du backend

Méthode	Endpoint	Description	Rôle
POST	/api/auth/register	Inscription	Public
POST	/api/auth/login	Connexion	Public
GET	/api/courses	Liste des cours	Tous
POST	/api/courses	Créer un cours	TEACHER
PUT	/api/courses/{id}	Modifier un cours	TEACHER
DELETE	/api/courses/{id}	Supprimer un cours	TEACHER
POST	/api/enrollments	S'inscrire à un cours	STUDENT
GET	/api/quiz/{id}	Détail d'un quiz	Tous
POST	/api/quiz	Créer un quiz	TEACHER
POST	/api/quiz/{id}/submit	Soumettre un quiz	STUDENT
GET	/api/exercises	Liste exercices	Tous
POST	/api/exercises	Créer un exercice	TEACHER
GET	/api/recommendations	Recommandations	STUDENT
GET	/api/search	Recherche de cours	Tous
GET	/api/admin/users	Liste utilisateurs	ADMIN